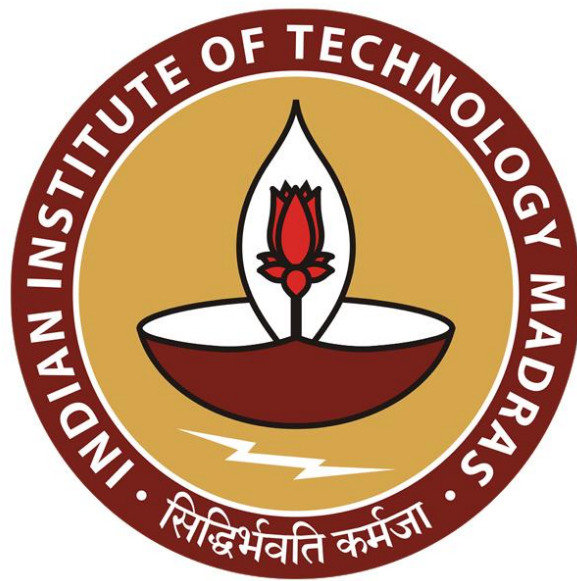


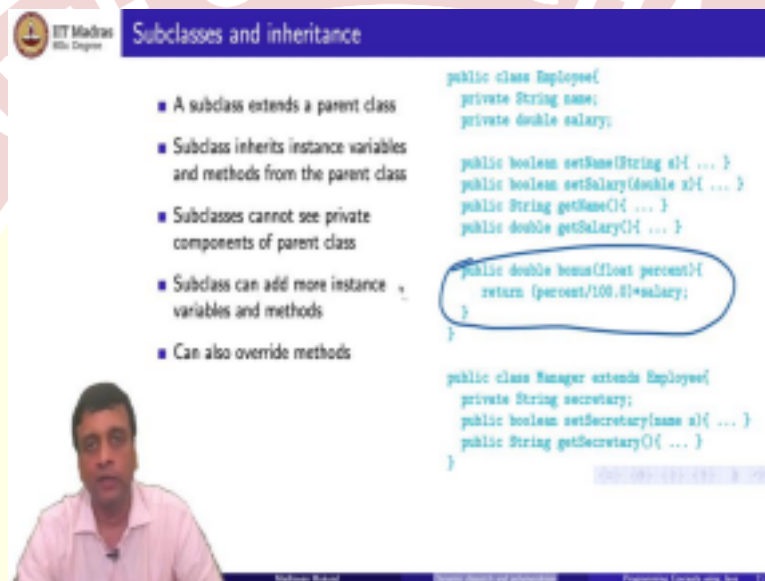


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Dynamic Dispatch and Polymorphism

(Refer Slide Time: 00:14)



Subclasses and inheritance

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclasses cannot see private components of parent class
- Subclass can add more instance variables and methods
- Can also override methods

```
public class Employee{
    private String name;
    private double salary;

    public boolean setName(String n){ ... }
    public boolean setSalary(double s){ ... }
    public String getName(){ ... }
    public double getSalary(){ ... }

    public double bonus(float percent){
        return (percent/100.0)*salary;
    }
}

public class Manager extends Employee{
    private String secretary;
    public boolean setSecretary(String s){ ... }
    public String getSecretary(){ ... }
}
```

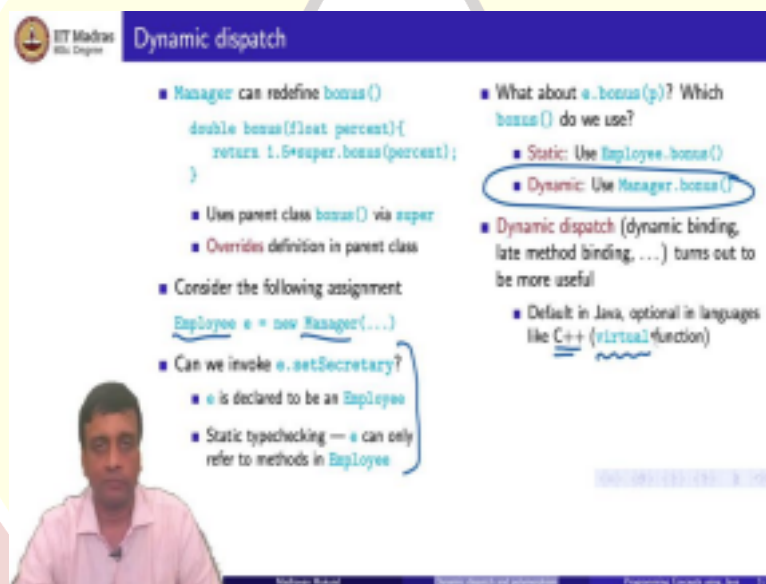
So, we saw that we can arrange classes in a hierarchy we can define a sub class that extends a parent class. So, this was the example we had before. So, we had an employee class and then we created a class called manager which extended the employee class. So, the subclass inherits the instance variables and the methods in the parent class. So, there are some instance variables for the name and the salary and there are these methods which set the name set the salary get the name get the salary.

And then there is this double bonus method which calculates the bonus given the percentage of bonus. So, whenever we create a manager object all these parts are implicitly there and in addition the manager object we said can add new instance variables and new methods. So, it is an extension of the employee class. The other thing we said was that these private components of the parent class are not visible to the sub class because in general we said the sub class may be used or implemented by somebody different from the person who created the parent class.

So, the only way that the child class can get access to these is through the public method. So, you can use these methods of course but also at the time of constructing you can use a super method to call the constructor of the parent class. So, though we have seen that a sub class can add more instance variables and methods what we did not see was that you can also change some of the methods that are given.

So, in particular look at this. So, we have a bonus which is calculated for employees and perhaps managers have a different type of bonus. So, maybe the way we want to compute a bonus for the manager is different from the way we want to compute the bonus for a normal employee. So, we should be able to write a different implementation of bonus which will override right it will override the method which is given by default from the parent class.

(Refer Slide Time: 02:12)



Dynamic dispatch

- Manager can redefine bonus()

```
double bonus(float percent){  
    return 1.5*super.bonus(percent);  
}
```

 - Uses parent class bonus() via super
 - Overrides definition in parent class
- Consider the following assignment

```
Employee e = new Manager(...)
```
- Can we invoke e.setSecretary?
 - e is declared to be an Employee
 - Static typechecking — e can only refer to methods in Employee
- What about e.bonus(p)? Which bonus() do we use?
 - Static: Use Employee.bonus()
 - Dynamic: Use Manager.bonus()
- Dynamic dispatch (dynamic binding, late method binding, ...) turns out to be more useful
 - Default in Java, optional in languages like C++ (virtual function)

So, here could be a definition of the manager's bonus. So, the manager's bonus has the same signature or this so it takes a float percentage as its input and it returns a double as the output. And notice now that we are not actually computing the managers bonus in terms of the salary if you go back you will see that for the employee the bonus was create computed as a percentage of salary.

But notice that the salary is a private variable of employee and since salary is a private variable of employee it is not visible to manager right. So, I cannot write here something like 1.5 times percentage by 100 times salary because salary is not available to me right.

So, this is not allowed but what I can do is I can call the bonus of the employee using again

this notion of super.

So, super in general talks about the function which is developed which is available one level up in the class hierarchy right. So, you go to the parents thing compute the bonus there take whatever result you get from that and add fifty percent to it. So, you multiply it by 1.5. So, this way we can get around the problem that a manager has no access to the primary variable salary which is required to compute the bonus.

And yet because employee has provided a way to do it you can kind of piggyback the new overridden function bonus in the manager class by using super to call the parent owner. So, now let us look at this definition right. So, we say employee e is a new manager and this is legal we said because every time we expect an employee we are ok with the manager because manager is a sub type right it is a sub class. So, think of set theory. So, every manager is an employee.

So now what about the instance variables, so we know that manager has this extra instance variable which is a secretary and it has an extra method to set the secretary name and to get the security name. So, can we take this employee which we know to be a manager right and invoke set secretary on it now the problem with this is that e in general is declared to be an employee.

So, as far as Java's type checking system goes the employee class has no set secretary method. So, if I call e.setSecretary statically it will be illegal right because e as a type does not have that method. So, static type checking will not allow this but of course we know that manager is actually the type of the object at runtime because that is how we have assigned it right.

So, how do we get that right. Now here what happens for instance if we take the same e and we call bonus on it remember now that there are two types of bonus available to us we had the original bonus which comes with an employee and we have overridden this by defining a bonus available with a manager right. So, we could potentially think of two ways. So, if we follow the static route that we use for disallowing set secretary.

Then in the static sense an employee variable should invoke the bonus which is defined in the employee class however if we are able to go past this and look at the dynamic status of the object then we would rather use the bonus in the manager. So, which of these do you think we will use? So, java actually uses this notion of dynamic dispatch. So, dynamic dispatch which is also called dynamic binding or late method binding essentially it waits to see what the runtime of the object is like.

And depending on the type of that runtime object it chooses the most appropriate function with respect to that runtime type. So, in this case at runtime it will see that e is actually a manager and manager has an overridden definition of bonus and therefore in this particular case what we will get is this version. So, notice that there is a subtle difference here. So, when I try to invoke a function which is not defined for the static type then I cannot define it even though dynamically is assigned to a more capable object which has that function.

But if I do have that function and there is a better version of that function then dynamic dispatch kicks in and I can use that version right. So, this is the default in java. So, in java essentially whether a function can be called or not depends on the static type which function is called depends on the dynamic type right. So, java uses dynamic dispatch by default not all object oriented languages do this.

So, in C++ for instance which is another popular object oriented language if you want this behavior you have to define that method or function to be something called a virtual function. If you do not call it a virtual function then the static function which is associated with the class which you have defined your variable to use okay will be used for that function for that variable.

So, virtual functions are explicitly declared in C++. In java every function behaves like this. So, every time we have a method it is dynamically taken based on the runtime type of the object.

(Refer Slide Time: 07:38)



- Every Employee in `emparray` "knows" how to calculate its bonus correctly!

```
Employee[] emparray = new Employee[2];
Employee e = new Employee(...);
Manager m = new Manager(...);

emparray[0] = e;
emparray[1] = m;

for (i = 0; i < emparray.length; i++){
    System.out.println(emparray[i].bonus(5.0))
}
```



So, this now allows us to do something interesting. So, supposing we define an employee array right. So, let us just have a simple employee array which has two elements in it and what we do is because every position in this array should be an employee object but we know that employees objects can point to manager objects employee are managers our employees remember.

So, I can create an explicit employee and an explicit manager and I can make the first element of the array point to the employee and the second elementary area point to the manager. So, now I have a kind of heterogeneous array right it is notionally an array of employees but one of them is an employee the other is manager now I run a loop right. So, I go from zero to one right and I print out the bonus for that employee given a 5% bonus.

So, what will happen here is because of dynamic dispatch right at run time for this particular employee for array one it will pick up the manager bonus whereas for the `emparray`, `emparray[0]` because that was already an employee it will pick up the employee bonus right. So, without doing anything explicit just by running over a loop and calling this function on each element in the array different elements in the array which are at different levels in this class hierarchy will automatically call the correct function using dynamic dispatch.

So, in some sense every employee in this array knows how to calculate its bonus correctly.

So, this is a power of dynamic dispatch that we can write this kind of a generic

loop without having to worry about you know specifically looking at the type of each object and then deciding how to invoke it.

(Refer Slide Time: 09:28)

Polymorphism

- Every `Employee` in `emparray` "knows" how to calculate its bonus correctly!
- Recall the event simulation loop that motivated Simula to introduce objects
- Also referred to as runtime polymorphism or inheritance polymorphism

```
Employee[] emparray = new Employee[2];
Employee e = new Employee(...);
Manager m = new Manager(...);

emparray[0] = e;
emparray[1] = m;

for (i = 0; i < emparray.length; i++){
    System.out.println(emparray[i].bonus(5.0))
}
```

Handwritten note: "Every Employee"

Now if you recall at the beginning when we discussed the motivation for object oriented programming we said that the language that first introduced object oriented programming was Simula. And the reason why Simula needed objects was because of this event simulation loop right. So, you create a queue consisting of the first event in your simulation list and then you remove the head of the queue simulate it and then it generates new events and you put it in the queue.

And our question was how do we have such a queue first of all right which was well typed but yet may hold many different types of events. And the second question was how do we make sure that the simulate `e` is not a complicated switch of cases right. I do not want to say if `e` is event of type one do this if `e` is event of type two do that. So, I do not want to explicitly try to tell you how to simulate different types of events.

So, this is where dynamic dispatch was used in simula and it has come down as one of the primary benefits of using this object oriented approach. So, we see it in java also so basically just like that employee array which had different types of employees in it here we have a queue which have different types of events in it. So, each of the events in the queue will be a sub type of some parent event type each of them will have its own simulate method which overrides the parent simulate method.

So, when I pick up an actual event from this queue and I simulate it, it will run the local version of simulate. So, that is how this dynamic dispatch works and this was really the motivation of introducing it in a language like Simula and that is why we use it by default in java as well. So, this type of dynamic dispatch is also known in the object oriented programming literature as polymorphism more specifically as runtime polymorphism or inheritance polymorphism because this comes as a result of the runtime behaviour of an object and through the inheritance hierarchy.

Now polymorphism is a word which talks if you have a little bit of understanding of the Greek origin of certain words. So, poly means many, right and morph refers to form. So, this is saying that the same function takes many forms and which form of the function you use is determined at run time based on the current objects position in the hierarchies hence runtime polymorphism or inheritance polymorphism right.

So, this is. So, dynamic dispatch is one way of saying technically how the function is chosen right. So, dynamically you choose that function which is most appropriate for the object but at a programming language level in object oriented literature this is sometimes referred to as inheritance or run time polymorphism.

(Refer Slide Time: 12:12)

Functions, signatures and overloading

- Signature of a function is its name and the list of argument types
- Can have different functions with the same name and different signatures
 - For example, multiple constructors
- Java class `Arrays` has a method `sort` to sort arbitrary scalar arrays
- Made possible by overloaded methods defined in class `Arrays`

```
double[] darr = new double[100];
int[] iarr = new int[100];
...
Arrays.sort(darr);
// sorts contents of darr
Arrays.sort(iarr);
// sorts contents of iarr

class Arrays{
    ...
    public static void sort(double[] a){...}
    // sorts arrays of double[]
    public static void sort(int[] a){...}
    // sorts arrays of int[]
    ...
}
```

So, runtime polymorphism is one situation where when I call a function it may behave differently based on the argument right in this case the context because it is not really the argument I invoke bonus on a manager object it does something I invoke bonus on an

employee object it does something else the other situation is when actually it depends on the argument in the sense of the signature right.

So, remember that the signature of a function is the types of its arguments the list of its input parameters and their types and the output type. So, we already said that we can have different functions which have the same name but with different signatures and we have seen this in the context of constructors right. So, we said you can have different constructors which take different numbers of arguments.

So, you just write each as a separate function and the one that matches the invocation will be used and this kind of overloading of functions is not possible in a language like python but it is available in java. So, in particular if you look at the built-in class that handles arrays in java right it has a static sort method which can sort arbitrary arrays of basic scalar types you know int double float and all that.

So, for instance you can create a double array of size 100 right and an int array of size 500 and then you can use the same sort function in arrays it is a static function. So, you can call it without having to do anything and you pass it in one case the double array and in one case the integer array. So, you call the same function with two different types of arguments now the way that this is working is not through inheritance polymorphism but overloading right.

So, if you look inside the arrays class then you will find that there are actually multiple definitions of sort. So, there is a sort with a signature double a there is another sort of the signature int array a. So depending on which one you call. So, this one will come and invoke this definition right and this one will come and invoke this definition. So, depending on which one you call the type of the argument the signature of the call will match the signature of the function definition. So, this is overloading.

So, this is a very different way in which the same function can apparently take on different forms and it is nothing to do with polymorphism. There are two completely different things which have the same effect you may imagine which is that the behavior of the function at runtime is determined by the types of the arguments or the context in the case of runtime

polymorphism.

(Refer Slide Time: 14:47)

Functions, signatures and overloading

- **Overloading:** multiple methods, different signatures, choice is static
- **Overriding:** multiple methods, same signature, choice is static
 - `Employee.bonus()`
 - `Manager.bonus()`
- **Dynamic dispatch:** multiple methods, same signature, choice made at run-time

```
double[] darr = new double[100];
int[] iarr = new int[500];
...
Arrays.sort(darr);
// sorts contents of darr
Arrays.sort(iarr);
// sorts contents of iarr

class Arrays{
    ...
    public static void sort(double[] a){...}
    // sorts arrays of double[]
    public static void sort(int[] a){...}
    // sorts arrays of int[]
    ...
}
```

Ex: e = new Manager()

So, the main thing to note about overloading is that the choice between the methods is static based on the signature right. So, we know in advance that this d array was of this type and therefore it will pick this type right. So, it is not a runtime decision at all on the other hand when we override a function right as we saw before then we have different methods with the same signature but the choice is static right.

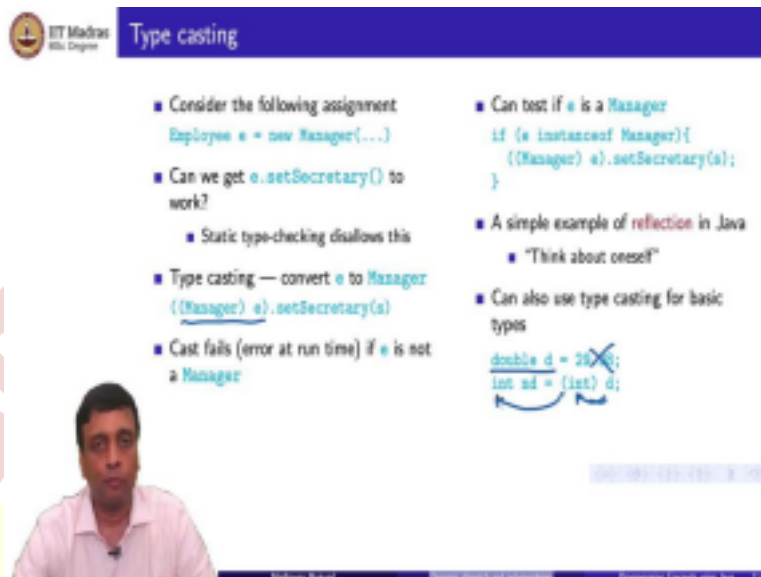
So, if we have an employee and we called it then its employee bonus right and if we have a manager and we call it, it is a manager bonus but the interesting case is when we have an employee variable pointing to a manager object right. So, you have a more specialized object being pointed to by a less specialized type variable at this point we have this polymorphism of dynamic this match right.

So, this is now a case where it is not based on the static type of the object but the dynamic type of the object. So, the choice of which function to call is made at runtime. So, you should be clear about all these things. So, if you have no confusion that is the type of the declaration matches the type of the object assigned to it then overriding will determine whether you are using the function which is defined in the current class or in the parent class.

If there is nothing in the current class you will inherit if you have something in the current class or if manager has redefined bonus then every manager object will call its

own bonus this is the case where we have employee e equal to some new manager right. So, then we have to do this.

(Refer Slide Time: 16:33)



Type casting

- Consider the following assignment
`Employee e = new Manager(...)`
- Can we get `e.setSecretary()` to work?
 - Static type-checking disallows this
- Type casting — convert `e` to `Manager`
`((Manager) e).setSecretary(s)`
- Cast fails (error at run time) if `e` is not a `Manager`
- Can test if `e` is a `Manager`

```
if (e instanceof Manager){
    ((Manager) e).setSecretary(s);
}
```
- A simple example of reflection in Java
 - "Think about oneself"
- Can also use type casting for basic types

```
double d = 25.5;
int id = (int) d;
```

So, now let us get back to the earlier case where we were asking about calling functions which we know are available dynamically but are statically not available. So, we said if we have employee e is equal new manager right and we want to use the set secretary function then static type checking we said will disallow this but supposing we are confident that this actually is available because we know that at this point this e is actually pointing to a manager.

So, how can we get java to accept this is the way to get java to accept this is to explicitly promote the type e right. So, this is called type casting. So, type casting takes a variable of one type and converts it to another type in this case if you are converting something and its now possible to do it, it will work. So, I take e and then this. So, I the way you cast is to put the type that you want in brackets before the thing.

So, this manager in brackets before the e right so this part. So, this converts the e which is an employee statically into a manager type and then now that is a manager type java is willing to accept that you can call this function set secretary what if it is not right. So, if you happen to invoke this cast at a time when it is not possible for example e is actually an employee then it is illegal to you cannot convert an employee to a manager because there are extra instance fields which have not been set.

So, an employee remembers that every manager is an employee but not every employee is a manager. So, this cast is not legal if you do not have this kind of an assignment before this. So, this would give you a run time error which is of course a bad thing. So, how can you avoid this runtime error? Because maybe; this is in a complex situation where you are not sure whether this is possible or not.

So, what you can do is you can actually query the status of the variable. So, you can ask whether at runtime this variable `e` is an instance of `manager`. So, currently what is `e` pointing to is it a `manager` or not right. So, this is a no is like a usual kind of status query it will tell you true or false and then you can say that if this is true if `e` is an instance of `manager` then and only then do you want to make this cast and set the secretary otherwise you will do something else.

So, this is an interesting thing that we can actually query the status. So, in python also you have this `type` function right you can say `type of x is something` right and you can find out the type of a variable at runtime. So, Java also allows this and in fact it allows more as we will see later on. So, this ability of a within the language. So, right. So, when you are running the program to queries the status of things which are artifacts of the program.

So, remember we are not asking about the value of `x` which is a normal thing we can we are not saying is `e` equal to something we are saying is the type of `e` equal to something. So, this is something about the program that is running rather than the state of the program. So, this kind of thing is called reflection. So, reflection in English means you know to think back and reflect not reflection in the sense of mirror.

But the philosophical kind of reflection right you sit back and reflect on your actions you reflect on your past. So, you reflect meaning you kind of think internally you introspect think about yourself. So, java allows a program to think about itself right. So, this is technically called reflection in programming languages and this is a very simple case of that this instance of predicate now type casting is not only for objects ok.

So, typecasting can actually also be used for the basic types that we introduced earlier which

are not objects like `float`. So, for instance you can take a variable of type `double` and then you can convert it to an `int` by using the type name `int` in a cast. So, casting can be used of course you should use it judiciously and you have to look up the java programming reference to understand what will happen.

For example when I convert `d` to an `int` presumably it will truncate in the sense that it will cut off this 0.98 and make it 29 but you have to check that this is the intended behavior before you do a cast right. And again if it at runtime this cast is illegal then you get a runtime error. So, you better be careful that you are doing it correctly. But here notice that there is no problem about it is a static thing right because for these kind of basic variables the type is known and there is no way to take a `double` and make it point to a `float` or make it point to an `int` every point a `double` is a `double` and `int` is a `int`.

So, this is really a static type checking thing that java can do it can check that this conversion is legal right and that it matches the left hand side and then once that is done it will do whatever is the appropriate type conversion of the value and store it. **(Refer Slide Time: 21:29)**

Summary

- A subclass can override a method from a parent class
- Dynamic dispatch ensures that the most appropriate method is called, based on the run-time identity of the object
- Run-time/inheritance polymorphism, different from overloading
 - We will later see another type of polymorphism, *structural polymorphism*
 - For instance, use the same sorting function for array of any datatype that supports a comparison operation
 - Java uses the term *generics* this
- Use type-casting (and reflection) overcome static type restrictions

IIT Madras
B.Tech. Degree

Dr. Ravi Kumar
Associate Professor
Department of Computer Science and Engineering
IIT Madras

so to summarize what we have seen is that not only can a sub class extend a parent class by adding new instance variables and new methods it can also override existing methods. So, in this case we saw that the manager class can override the bonus function of the employee class. Then we have this interesting situation that because of this compatibility you can have a variable of the parent class pointing to an object of the child class.

So, the child has more capabilities, so now if I call this overwritten function in the context of the parent class if I have an employee which is pointing to a manager object. If I ask for the bonus then the question is which bonus do I get and dynamic dispatch will actually use the runtime value. So, in this case we saw that if the current employee *e* is pointing to a manager then you will get the manager bonus. So, this is called runtime or inheritance polymorphism.

So, this is different from overloading right. So, overloading is based on signatures. So, this is a static thing it says that you can write the same function name with different signatures different arguments. We saw it for constructors we saw in the arrays the sort function which takes different types of basic type arrays double arrays and arrays and so on. So, based on the static type of the argument it will choose the correct definition among all of these which has the same name.

So, as the person using the function it looks like you have one magic sort function which will sort arrays of any type but actually you do not have one sort function right you have a separate sort function defined for every type and based on the static type of the argument java is picking the right one right. So, overloading is a static choice of functions among different signatures whereas dynamic dispatch is really based on the runtime type of the object.

So, there is another type of polymorphism which we shall see which is very useful right. So, imagine that let us get back to the sorting. So, what is it that is important to sort an array. So, essentially the sorting of an array requires us to put it in some order either ascending order or descending order. So, what is really important when we sort an array is not the values themselves.

But how to compare these values I need to know whether the value at position *i* in an array is smaller than bigger than or equal to position *j*. So, so long as I can compare if I have a comparison function between objects in an array it does not really matter whether these objects are integers or strings or you know more I could have a way of comparing dates for example right.

A date can be compared in terms of the day month and year or I could even compare lists in terms of their length. So, there could be any different many different ways of calling an array

of objects. So, long as we have this one common feature that I can compare. So, this is something called structural polymorphism now I can use a single sort function and this sort function will ask that its argument have a certain capability namely that capability of being compared.

And so long as the argument has the capability of being compared the same sort function will work without having to worry about what the exact object is right. So, this is called sometimes called structural polymorphism and this is quite common in say for example functional programming java also has this and it uses the term generics. So, we will see this later. So, there is its third way of doing polymorphism right.

So, there is one way of doing polymorphism which is just the kind of trick which is overloading there is another way which is based on dynamic dispatch which is based on the class hierarchy. And the third way is to use actually some property of the type and say that if the type has this property. If it can be compared if you can do something else then my function will work.

And this is something we will see later and the last thing we saw is that you can use type casting and some introspection you can look at the type to determine whether typecasting will work or not and use this to overcome some static type restriction. So, for instance if you want to set an instance variable of a more specific type from a less specific variable name you can do that by using casting.